

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code: CSA0305 Course Title: Data Structures
CO Assessed: CO4 – Articulation (BL3) Assessment: Assessment Tool 3 – Comparative Analysis
Weightage: 30% of CIA Total Marks: 100
CO4 Statement: Apply hashing strategies for efficient data storage and sorting algorithms for arrangement of data.
Student Name: Sumetha Reg. No.: 192521097
Section / Batch: Date: 26/08/2026

Code of Conduct

I Sumetha Sri S / 192521097 (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Sumetha

Instructions

- This test contains 5 Comparative Analysis questions. Total: 100 Marks.
- When a student answer using comparative analysis should be based on four requirements: time complexity, space complexity advantages & limitations, real-world scenarios and operations.
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Hashing	Linear Probing & Separate Chaining	1	20
Sorting	Merge Sort & Quick Sort	2	20
Hashing	Quadratic Probing and Double Hashing	3	20
Sorting Algorithms	Complexity Analysis	4	20
Quick Sort	Recursive and Iterative implementations	5	20
GRAND TOTAL:			100 Marks

Annexure A - Comparative Analysis

- Compare Linear Probing and Separate Chaining for collision handling in hashing based on: (20 Marks)
 - Clustering effect
 - Memory usage
 - Performance under high load factor
 - Which method is more suitable for large-scale applications and why?
- Compare quick sort and merge sort for sorting a large linked list of 10 million nodes. Analyze memory requirements (in-place vs. extra space), cache behavior, and time complexity. Which algorithm is preferable for linked lists and why? How does the answer change for arrays? (20 Marks)
- Compare Quadratic Probing and Double Hashing in terms of: (20 Marks)
 - Clustering issues
 - Collision resolution efficiency
 - Suitability for dense hash tables
- Compare Best-case and Worst-case time complexity of sorting algorithms based on the following: (20 Marks)
 - Practical significance
 - Impact on algorithm selection
- Compare Recursive and Iterative implementations of Quick Sort in terms of: (20 Marks)
 - Space complexity (stack usage)
 - Ease of implementation
 - Risk of stack overflow

Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Concept Understanding	20	Demonstrates complete and accurate understanding of all data structures with advanced insights	Shows clear understanding with minor gaps	Basic understanding; some misconceptions present	Limited or incorrect understanding
Comparative Analysis Depth	20	Thorough, multi-dimensional comparison with strong justification and critical evaluation	Good comparison with relevant factors considered	Limited comparison; lacks depth or justification	Minimal or incorrect comparison
Use of Parameters (Time/Space/Operations)	30	All parameters analysed accurately with complexity discussion	Most parameters covered correctly	Few parameters considered; some inaccuracies	Parameters missing or incorrect
Critical Thinking	20	Strong justification of why one structure is better in given contexts	Some justification present	Limited reasoning	No critical reasoning
Clarity	10	Exceptionally well-structured, logical flow, clear tables/diagrams	Well-organized with minor issues	Some organization issues; lacks clarity	Poor structure and readability

Faculty In-charge
Signature: _____
Date: _____

Course Coordinator
Signature: _____
Date: _____

Program Director
Signature: _____
Date: _____

CO4 - AT3 - COMPARATIVE ANALYSIS

1. Linear probing Vs Separate chaining

Parameter	Linear Probing	Separate chaining
Basic Principle	Finds the next empty slot in the hash table	Stores collided elements in a linked list / bucket
Clustering effect	High primary clustering	Very low clustering
Memory usage	Low; no extra nodes / pointers	Higher, requires extra nodes / pointers
Load factor	Must generally be < 1	Can be > 1
Average search	$O(1)$ at low load factor	$O(1 + \lambda)$
Worst-case search	$O(n)$	$O(n)$
Insertion	Very fast at low load factors	Fast when chains are short
Deletion	More complicated; tombstones may be needed	Relatively simple.
Cache locality	Excellent	Lower

High-load performance	Degrades rapidly	Degrades more rap
Implementation	Simple	Moderately complex
Range-scale applications	Suitable when memory/cache efficiency is important	More suitable for dynamic/high-load applications
Overall advantage	Memory efficient	Better collision and high-load handling

2. Quick Sort Vs Merge Sort.

Parameter	Quick Sort	Merge Sort
Best-case time	$O(n \log n)$	$O(n \log n)$
Average-case time	$O(n \log n)$	$O(n \log n)$
Worst-case time	$O(n^2)$	$O(n \log n)$
Space complexity	$O(\log n)$ average	$O(\log n)$ auxiliary recursion, merging can use links
Random access	Not suitable for linked lists	Not required
Partitioning	Less efficient with linked lists	Not needed in the same way.
Merging	Not its strength	Very efficient using pointers

Stability	Usually not stable	Stable
Risk for 10 million nodes	Higher due to possible $O(n^2)$	Lower and predictable
Linked-list Suitability	Moderate	Excellent
Implementation for linked list	More difficult to optimize	Relatively straight forward.
Memory movement	Can require pointer/data rearrangement	Nodes can be rearranged using links
Overall choice	Not preferred	Preferred

3. Quadratic Probing Vs Double Hashing

Parameter	Quadratic Probing	Double Hashing
Basic principle	Uses quadratic increments to find next position	Uses a second hash function to determine probe step.
Formula	$h(k) + c_1 i + c_2 i^2$	$ch_1(k) + i \times h_2(k)$
Primary clustering	Reduced significantly	Very low

Secondary clustering	Possible	Very low
Collision Efficiency	Good	Better
Distribution of Keys	Moderately distributed	More uniformly distributed
No. of Hash functions	One	Two
Implementation	Simplex	More complex.

4. Best case Vs Worst case of Sorting Algorithms

Parameter	Best case	Worst case
Meaning	Minimum execution time	Maximum execution time
Input condition	Favorable input arrangement	Unfavorable input arrangement
Performance	Fastest possible	Slowest possible
Practical Significance	Shows potential difference	Shows performance guarantee under difficult input
Quick sort	$O(n \log n)$	$O(n^2)$
Merge sort	$O(n \log n)$	$O(n \log n)$
Insertion sort	$O(n)$	$O(n^2)$

Algorithm Selection	Useful for favorable / typical situations	Important when predictable performance is required.
Risk	May not represent difficult inputs	Indicates maximum possible running time.
Importance	Helps understand optimization	Very important for reliable systems

5. Recursive Quick Sort Vs Iterative Quick Sort.

Parameters	Recursive Quick Sort	Iterative Quick Sort
Implementation	Uses recursive function calls	Uses an explicit stack.
Space-Average Case	$O(\log n)$	$O(\log n)$ with controlled stack
Space-Worst Case	$O(n)$	Can be $O(n)$, depending on implementation
Stacks usage	Uses system call stack	uses programmer-controlled stack.
Ease of implementation	Easy	More complex
Code complexity	Shorter and simpler	Longer and more complicated

Stack overflow risk	Higher for unbalanced partitions	Lower with proper stack management
Memory Control	Less direct	More control
Large Input	Can cause deep recursion	Better suited for large input
Performance	Good	Comparable
Advantages	Simple and easy to understand	Better control of stack memory

Wrote